MODULE 19 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

Reading a Query Profile

The module most likely to move a borderline score. Four numbers answer almost every question.

DOMAIN 4 of 5 WEIGHT 21% MODULE 19 of 31

admin@sthithapragna.com

The profile does not tell you the query is slow. It tells you **which of four things** is wrong.

Finding it, and what is in it

- In Snowsight: **Activity, then Query History**, select a query, then the **Query Profile** tab.
- From a worksheet, there is a **Query Profile link** beside the results.
- Four elements: **Steps**, the **operator tree**, the **node list**, and the **overview**.
- The operator tree is a **DAG**. Darker nodes consumed more resources.

> ON THE EXAM
>
> The profile is available for any query in history, not just one you just ran. That matters for investigating last night's job.

How to read the tree

- Read it **bottom-up**. Leaf scans feed joins, joins feed aggregates, and the top feeds **Result**.
- **Time in a node includes its children**, so the percentages do not sum to 100. Read them as which subtree dominates.
- **Arrows carry row counts.** Look for an operator emitting far more rows than it consumes.
- **Most Expensive Nodes** lists operators taking at least **1%** of execution time. Start there.

> ON THE EXAM
>
> Most Expensive Nodes is the first thing to look at. If the top node is a TableScan, the problem is data volume or pruning.

Execution time breakdown

CATEGORY	WHAT DOMINANCE MEANS
Processing	CPU-bound. The query may simply be big. Filter earlier, or pre-aggregate.
Local Disk I/O	The working set did not fit in memory and paid a disk penalty.
Remote Disk I/O	Blocked on cloud storage. A huge scan, or spilling past local disk. The alarm bell.
Network Communication	Data movement between nodes.
Synchronization	Workers waiting on each other. Often the fingerprint of skew.
Initialization	Overhead of starting the query on the warehouse.

ON THE EXAM

For most queries the time is dominated by Processing or IO. Large Network or Initialization suggests a non-obvious problem.

The statistics that answer the four questions

STATISTIC	WHAT IT TELLS YOU
Partitions scanned, partitions total	Pruning. Close to equal means you are reading almost everything.
Bytes spilled to local storage	The query exceeded memory. Bad.
Bytes spilled to remote storage	It exceeded even local disk. Dramatically worse.
Percentage scanned from cache	How much came from the warehouse local disk cache.
Rows in and rows out on each arrow	Join behaviour. Out far exceeding in is the exploding join signal.

ON THE EXAM

Five numbers. If you learn to find these quickly, you can diagnose almost any profile question on the paper.

Problem one: inefficient pruning

- **Partitions scanned close to partitions total**, on a query that filters to a small slice of the table.
- The most expensive node is usually a **TableScan**, and Remote Disk I/O is high.
- Remedies: a **clustering key** on the filter column, a more selective filter, or fewer columns selected.
- **A bigger warehouse does not fix this.** It reads the same excess data faster and costs more.

> ON THE EXAM
>
> Watch for JoinFilter. Snowflake can derive a range from one side of a join to prune the other, so good pruning may appear without a WHERE clause.

Problem two: queries too large to fit in memory

- **Bytes spilled to local storage**, and worse, **bytes spilled to remote storage**. Local Disk I/O or Remote Disk I/O dominant.
- The expensive node is often a **Sort**, a **Join** or an **Aggregate**. Those are the memory-hungry operators.
- Documented remedies, in order: **size up** for more memory, **optimise the query**, **reduce data scanned** with clustering.
- Snowflake's own guidance: **resizing should come after optimising the workload**, not instead of it.

> ON THE EXAM
>
> Remote spilling is the single clearest scale-up signal in the whole profile. Local spilling is a warning; remote is an emergency.

Problem three: exploding joins

- A **Join node emitting far more rows than it consumed**, and usually topping the expense list.
- Cause: a **missing join condition**, producing a Cartesian product, or one that is **too loose**.
- Check the arrows. Two inputs of a few million rows producing billions is not a compute problem.
- **No warehouse size fixes this.** The query is producing rows that should never have existed.

> ON THE EXAM
>
> This is the one where scaling up is most tempting and most wasteful. The row counts on the arrows give it away immediately.

Problem four: UNION without ALL

- **UNION** eliminates duplicates. **UNION ALL** does not, and is therefore much cheaper.
- In the profile it appears as a **UnionAll operator with an unexpected Aggregate on top**. That Aggregate is doing the deduplication.
- If duplicates are impossible by construction, **UNION ALL** removes that work entirely.
- The cost is invisible in the SQL. Three characters separate a cheap query from an expensive one.

> ON THE EXAM
>
> The clue is an Aggregate operator you did not ask for. Nothing in the SQL says GROUP BY, and there it is.

Symptom, node, remedy

WHAT YOU SEE	MOST LIKELY NODE	WHAT TO DO
Partitions scanned near total	TableScan	Clustering key, or a more selective filter
Bytes spilled to remote storage	Sort, Join or Aggregate	Scale up. Then optimise the query
Rows out far exceeding rows in	Join	Fix the join condition. Do not resize
An Aggregate nobody asked for	UnionAll	Use UNION ALL if duplicates are impossible
Queued time before execution	None. It had not started	Scale out, or reduce concurrency
High Synchronization	Any	Skew. Workers waiting on one overloaded colleague

ON THE EXAM

Six rows covering the module. Rows 1 to 4 are Snowflake's four documented problems, in order.

> A dashboard query filtering to a single account_id over a 9 TB events table takes 90 seconds. Query Profile shows: TableScan is 81% of execution time; partitions scanned 118,400 of 121,000; bytes spilled to local storage 0; bytes spilled to remote storage 0; no queued time.
>
> Remote Disk I/O dominates the time breakdown.

What is the problem, and what should be done?

A. Pruning is failing. The query reads almost every partition to return one account. Cluster on account_id, or filter more selectively.

B. The warehouse is too small. Remote Disk I/O dominance means it needs more memory.

C. An exploding join. TableScan at 81% indicates rows being multiplied downstream.

D. The result cache is not being used. Enable USE_CACHED_RESULT for the dashboard's role.

◆ PAUSE AND DECIDE

What is the problem, and what should be done?

A Pruning is failing. Cluster on account_id.

118,400 of 121,000 partitions is 98% of the table, to return one account. TableScan dominant, and both spill figures are zero.

B Too small. Remote Disk I/O means more memory.

Remote Disk I/O has two causes: spilling, or a huge scan. Both spill figures are zero, so this is the scan.

C An exploding join.

A TableScan is not a join. The scenario gives no row counts suggesting multiplication, and mentions no join at all.

D Enable the result cache.

It is on by default, and a dashboard filtering by varying account_id would rarely repeat a query exactly anyway.

THE SIGNAL

Both spill figures being zero is what separates A from B. Remote Disk I/O without spilling means the query is reading too much.

> A nightly aggregation on a Medium warehouse takes 3 hours. Query Profile shows: Sort is 64% of execution time; bytes spilled to local storage 890 GB; bytes spilled to remote storage 210 GB; partitions scanned 4,100 of 96,000; no queued time.
>
> Local Disk I/O and Remote Disk I/O together dominate the time breakdown.

What is the problem?

A. Inefficient pruning. Add a clustering key to reduce the partitions scanned.

B. Concurrency. Convert the warehouse to multi-cluster with a maximum of four.

C. The query exceeded memory and spilled, including to remote storage. Scale up, then optimise the query.

D. UNION without ALL. The Sort operator is performing duplicate elimination.

◆ PAUSE AND DECIDE

What is the problem?

A Inefficient pruning. Add a clustering key.

4,100 of 96,000 is about 4%. Pruning is working well. This is the number that rules the option out.

B Concurrency. Go multi-cluster.

No queued time, and one query cannot use two clusters. Nothing here is a concurrency problem.

C Memory exceeded, spilling to remote. Scale up, then optimise.

890 GB local and 210 GB remote spill, with Sort dominant and IO dominating the breakdown. The textbook signature.

D UNION without ALL.

That appears as an Aggregate above a UnionAll, not as a Sort. No union is mentioned.

THE SIGNAL

The pruning figure is the deliberate distractor. It looks like a big number until you notice it is 4% of the table.

> A report joining an orders table to a shipments table has started taking hours. Query Profile shows a Join node at 91% of execution time. The arrows show 2.4 million rows entering from one side, 6.1 million from the other, and 14.6 billion rows leaving the join.
>
> Partitions scanned is 6% of total. Bytes spilled to remote storage is large.

What is the root cause?

A. Memory. The remote spilling is the primary problem, so the warehouse should be scaled up.

B. Pruning. 6% is too low, so the join is reading stale partitions.

C. Skew. One worker received most of the rows, which is why the Join node dominates.

D. An exploding join. 2.4 million by 6.1 million producing 14.6 billion means the join condition is missing or too loose.

◆ PAUSE AND DECIDE

What is the root cause?

A Memory. Scale up.

The spilling is real and it is a symptom. 14.6 billion rows will exhaust any warehouse. Fix the cause, not the consequence.

B Pruning. 6% is too low.

6% is excellent pruning. Low is good here, and the option inverts the meaning of the ratio.

C Skew.

Skew shows as high Synchronization time, not as row multiplication. The row counts point elsewhere.

D An exploding join. The condition is missing or too loose.

Rows out exceed rows in by roughly a thousandfold. That is the documented signature, and no warehouse size fixes it.

THE SIGNAL

The spilling is a consequence, not the cause. Scale up and you will spill less while still producing 14.6 billion unwanted rows.

> A query combines twelve monthly partitions of the same table with UNION between each pair. Each month's rows are distinct by construction; no row can appear in two months.
>
> Query Profile shows UnionAll operators, and above them an Aggregate operator taking 47% of execution time. The SQL contains no GROUP BY and no DISTINCT.

What is the Aggregate doing, and what should change?

A. It is computing statistics for the optimiser. This is normal overhead and cannot be avoided.

B. UNION eliminates duplicates, and that Aggregate is doing it. Use UNION ALL, since duplicates are impossible here.

C. It is the result cache being populated. Set USE_CACHED_RESULT = FALSE to remove the overhead.

D. It is performing the partition pruning for all twelve scans. Add a clustering key to remove it.

◆ PAUSE AND DECIDE

What is the Aggregate doing, and what should change?

A. Optimiser statistics. Unavoidable.

Statistics come from micro-partition metadata, not from a runtime operator taking 47% of the query.

B. UNION deduplicates. Use UNION ALL.

The documented signature: a UnionAll with an unexpected Aggregate above it doing duplicate elimination. Three characters remove 47% of the work.

C. The result cache being populated.

Populating the cache is not an operator in the tree, and disabling it would make repeat runs slower, not faster.

D. Pruning for the twelve scans.

Pruning happens at the TableScan using metadata, before any data is read. It never appears as an Aggregate.

THE SIGNAL

'No GROUP BY and no DISTINCT' is the whole clue. An Aggregate you did not ask for is doing work you did not know you requested.

What to carry into the next module

- **Start at Most Expensive Nodes.** TableScan means volume or pruning. Join means the join. Sort means memory.
- **Partitions scanned against total** is pruning. Low is good. **Bytes spilled remote** is the clearest scale-up signal.
- **Rows out far exceeding rows in** at a Join is an exploding join. No warehouse size fixes it.
- **An Aggregate you did not ask for**, above a UnionAll, is UNION deduplicating. Use UNION ALL.
- **Time in a node includes its children**, so percentages do not sum to 100.

> ON THE EXAM
>
> Next: clustering, materialized views, search optimization and query acceleration. Four features that all make queries faster.